Progressive Retrieval Attention with AirLLM: Independent Weight and Semantic-Context Streaming

Jorge Simão

August 2026

Abstract

AirLLM makes models larger than accelerator memory executable by loading one decoder layer, or selected experts, immediately before use and releasing its weights afterward. Progressive Retrieval Attention (PRA) addresses a different state variable: it routes over persistent context and materializes only selected native key/value (K/V) detail. We ask whether these two forms of virtualization compose without conflating model-weight and semantic-memory policy. We audit AirLLM 3.3.0 at commit `3099999`, implement a capability-gated runtime provider, reuse the existing Hugging Face PRA attention path, and introduce a layer-local lifecycle adapter with HOT, layer-streamed, and hybrid PRA residency. On Apple M5, AirLLM's separate MLX path executes TinyLlama-1.1B from 2.20-GB layer shards. Reducing a 653-token prompt to 38 selected tokens preserved the 12-token answer prefix, but throughput remained about 0.13 token/s because weight streaming dominated. In a 119-condition controlled lifecycle study, layer-streaming reduced balanced-profile PRA peak residency from 0.50 to 0.125 MiB; independent prefetch was faster than sharing AirLLM's one-worker weight executor. On a 4-GiB GTX 950M, injected-but-disabled PRA preserved the exact AirLLM output, and native PRA encoded and consumed a 15-token reference through four layers. Native decode was 10.6% slower than selected-text E0, so this establishes mechanism correctness rather than a speed advantage. The evidence supports independent policy and exposes the central systems tradeoff: eliminating context residency is valuable only when its transfers do not deepen the weight-I/O bottleneck.

1 Introduction

Inference memory is not one pool. Model parameters, sequential K/V, selected external K/V, temporary activations, and runtime allocations have different lifetimes and reuse patterns. AirLLM [1] attacks parameter residency by building a model skeleton on the meta device and streaming checkpoint shards through forward hooks. PRA attacks context residency: a query selects bounded regions from typed resources, and designated decoder layers consume their native K/V without exposing the whole resource as sequential prompt text.

This combination is appealing but not automatic. The same storage device and host-to-device path may carry both layer weights and PRA detail. Keeping PRA K/V HOT consumes memory deliberately freed by AirLLM. Streaming PRA K/V saves that memory, but adds another demand stream. Moreover, replacing an HF attention module changes its parameter path even when all pretrained tensors are reused, which can invalidate AirLLM's shard loader.

We study one question:

> Can model-weight streaming and query-addressed semantic-context streaming be controlled independently while still composing at an attention layer?

The paper makes four scoped contributions. First, it records a source-grounded AirLLM capability audit. Second, it adds a conservative AirLLM provider to the shared PRA runtime: selected-text E0 is the default, while native E1/E2 must be requested explicitly. Third, it implements the parameter-name and device bridge needed to reuse the HF-native PRA path, plus a layer-local K/V lifecycle adapter. Fourth, it reports live Apple/MLX selected-text evidence and controlled residency, prefetch, and memory-frontier measurements. We distinguish measured live results, controlled-model results, source-audit conclusions, and pending CUDA evidence.

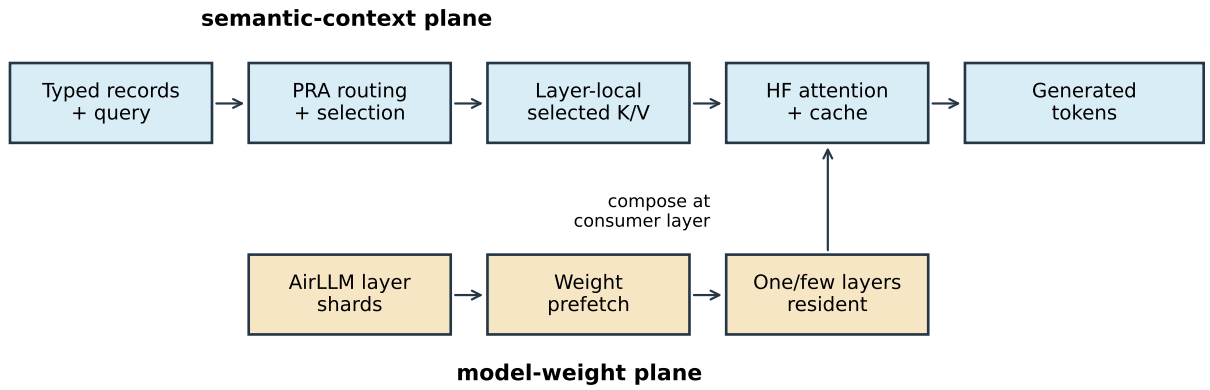

Figure 1: AirLLM owns weight-shard movement. The shared PRA runtime owns record identity, routing, authorization, and selected K/V. Their physical lifecycles compose at an HF attention consumer without sharing semantic policy.

Table 1: AirLLM mechanisms and PRA consequences at the pinned revision.

Mechanism	Source-audited behavior	PRA consequence
HF execution	`self.model.generate` and `self.model` own generation/forward	Reuse HF PRA on CUDA/Linux
Model skeleton	Built under Accelerate meta initialization	Request device cannot be inferred from embedding residency
Weight lifecycle	Module pre-hook loads; post-hook returns weights to meta	Attach PRA lifecycle after weight hook
Prefetch	One-worker thread executor	Shared scheduling can serialize two reads
Pinned host memory	Per-prefetched-layer bound	PRA pinning needs a separate explicit budget
Sparse experts	Routed expert pre/post hooks when topology permits	Expert and context selection remain independent
macOS path	Separate `AirLLMLlamaMlx` implementation	E0 only; do not claim HF-native PRA

2 Two Independent State Planes

Figure 1 separates the semantic-context plane from the model-weight plane. The systems meet only when a designated consumer layer is resident and its selected K/V is active.

We account for peak memory as

$$M_{\text{peak}} = M_{\text{weights,hot}} + M_{\text{localKV}} + M_{\text{PRA,hot}} + M_{\text{temporary}} + M_{\text{framework}}. \tag{1}$$

PRA WARM/COLD bytes are reported separately because they are retained physical state but not accelerator working-set bytes. This decomposition matters: once AirLLM removes full-model residency, local K/V may become the dominant term.

3 AirLLM Audit

We pin AirLLM 3.3.0 at commit `309999931a3cc5572d8880e7e0f18d408f3a067b`. The audit script records source files and line numbers rather than inferring behavior from package names. Table 1 summarizes the resulting contract.

The source audit found HF delegation in `airllm_base.py`, lines 904–907; streaming-hook installation in the same file, lines 763–764; and expert hooks near line 834. The Mac dispatcher enters a separate MLX class in

`auto_model.py:7` and does not expose AirLLM's HF `.model`. Consequently, platform capability is not uniform: the Mac run validates AirLLM weight streaming and E0 selected text, not native PRA.

4 Integration

4.1 Capability-Gated Provider

The shared provider registry now includes `airllm`. It advertises E0 by default. Setting `native_hf_pra=true` enables E1 only on the HF-backed path; adding `storage_managed=true` advertises E2. The provider is embedded-only because AirLLM is a Python model wrapper rather than an HTTP scheduler. This prevents the CLI and product matrix from presenting the MLX path as native merely because the package imports.

4.2 Reusing HF Attention

PRA wraps selected `self_attn` modules while retaining each original attention module as `original_attention`. AirLLM shards still contain ordinary keys such as

`model.layers.7.self_attn.q_proj.weight`

The adapter maps that key, only for PRA-injected layers, to

`model.layers.7.self_attn.original_attention.q_proj.weight`

at placement time. Checkpoint files are neither rewritten nor duplicated. Unselected attention and all non-attention keys are unchanged. The request device is taken from AirLLM's `running_device`; relying on an embedding parameter would incorrectly return `meta` when embeddings are streamed.

In simplified form, the native bridge is:

```
pra = PRAForCausalLM.from_model(airllm.model, airllm.tokenizer)
for shard_key, tensor in layer_shard:
    target = insert_original_attention(shard_key) if pra_layer else shard_key
    airllm.place(target, tensor)
pra.request_device = airllm.running_device
```

AirLLM continues to invoke Transformers. PRA continues to use the model's own Q/K/V projections, RoPE implementation, and cache semantics.

4.3 Layer-Local PRA Lifecycle

The lifecycle adapter registers its hooks after AirLLM. PyTorch therefore runs:

```
AirLLM pre: load W_l
PRA pre:    activate selected (K_l, V_l)
forward:    local attention + selected-memory attention
AirLLM post: release W_l
PRA post:   release (K_l, V_l), unless policy keeps it HOT
```

The adapter supports three residency modes: HOT keeps every selected consumer layer resident; `layer_streamed` retains only the active layer; `hybrid` keeps an explicit layer subset HOT. Closing an adapter releases all active detail, enforcing the invariant that a later request cannot inherit another request's selected memory.

5 Experimental Method

5.1 Evidence Tiers

- Source audit: pinned source and runtime-import facts.
- Live engine E0: actual AirLLM/MLX model loading and generation.
- Controlled model: measured hooks, stores, transfers, and memory accounting with a deterministic eight-layer execution model; no language quality.
- Live native smoke: CUDA HF-backed mechanism and parity tests.

Table 2: Live AirLLM/MLX E0 generation. One run per condition; 12 greedy output tokens. TTFT is first yielded token, not HTTP serving TTFT.

Condition	Input tok.	TTFT (s)	Total (s)	tok/s	Output prefix
Full, short	185	7.65	89.18	0.135	`The project access code is CYAN-ORBIT-`
Selected	38	7.60	93.00	0.129	same
Full, longer	653	8.27	90.23	0.133	same
Selected	38	7.99	88.97	0.135	same

Table 3: Live AirLLM/HF execution on a 4-GiB GTX 950M. Four greedy output tokens; peak is PyTorch CUDA allocation, not total device use.

Condition	Visible tok.	Native tok.	Total (s)	Peak (MiB)
Full-context E0	377	0	40.29	160.42
Selected-text E0	38	0	28.95	135.20
PRA injected, disabled	38	0	31.35	135.20
Native PRA	19	15	32.03	135.63

5.2 Live Apple Run

The live E0 run used an Apple M5 with 16 GB unified memory, Python 3.11, AirLLM 3.3.0, MLX 0.32.2, Transformers 5.12.1, and `TinyLlama/TinyLlama-1.1B-Chat-v1.0`. AirLLM produced 50 shard/marker files totaling 2,200,159,150 bytes. We froze one evidence sentence and compared it as selected text against prompts containing repeated irrelevant archive notes. Greedy decode emitted 12 tokens.

5.3 Controlled Lifecycle Sweep

The controlled model has eight decoder layers, four K/V heads, 64-dimensional heads, FP16 K/V, 128 selected tokens, and 128 local query tokens. Each streamed weight layer is modeled as 8 MiB over an 800-MiB/s store with 3 ms of layer compute. We crossed backing contexts of 2K, 8K, 32K, and 64K tokens; three consumer profiles; three residency modes; and three prefetch policies, then added WARM, COLD, and SOURCE restoration controls. The result contains 119 rows with three repeats for timed native conditions.

6 Results

6.1 Selected Text on AirLLM/MLX

Selected text reduced visible input by 79.5% relative to the 185-token prompt and by 94.2% relative to the 653-token prompt. It preserved the complete 12-token output prefix in all four conditions. The run stopped one generated token before the final digits of the synthetic code, so we report prefix agreement rather than exact-answer accuracy. Cold initialization, including download/sharding, took 109.6 s; reuse of existing shards initialized in 0.57 s. The first instrumented run observed about 1.10 GiB maximum change in system available memory, while process RSS grew from 346 to 832 MiB. System-available memory is noisy on a shared host and is not treated as process peak memory.

The small TTFT and throughput differences are the main result, not a failure of selection. Each decoded token streams the model layers again. At these prompt lengths, reducing sequential context cannot remove that fixed weight-I/O floor. This finding motivates native persistent K/V chiefly for repeated queries and longer backing contexts, not as a universal latency improvement.

The same run also exposed an AirLLM/MLX compatibility boundary: SmolLM2-135M uses tied embeddings and had no independent `lm_head` shard, but the MLX generation path attempted to load one. We retain this blocked run as an engine/model-family finding and do not count it as PRA evidence.

6.2 HF-Backed CUDA Correctness

The full and selected E0 conditions generated the same four-token prefix. Selected text removed 89.9% of visible input, reduced completion latency by 28.2%, and reduced peak allocated CUDA memory by 15.7%. After PRA in-

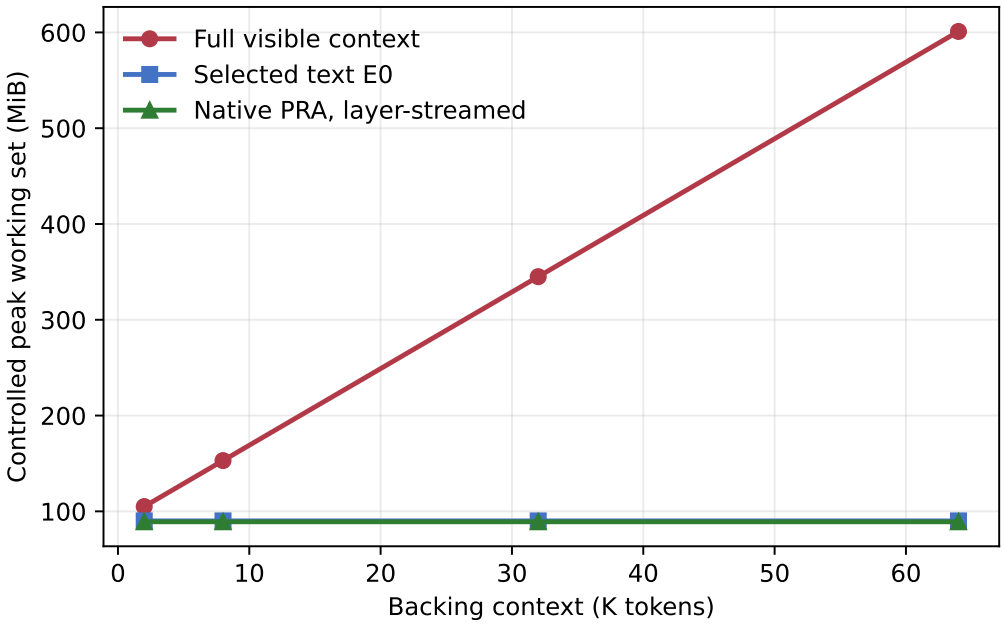

Figure 2: Controlled working-set decomposition. Full-context local K/V grows with backing length; selected text and native selected K/V remain bounded. These values are model-derived accounting, not GPU telemetry.

jection, the disabled condition matched the selected E0 token sequence exactly. This is the key parameter-placement result: every streamed decoder shard reached the PRA-wrapped path without changing disabled semantics.

Reference encoding took 7.12 s for 15 tokens. Native generation exposed only 19 query tokens sequentially and materialized all 15 selected K/V tokens at the configured final four consumer layers. It generated the same prefix in 32.03 s and peaked at 135.63 MiB. Relative to selected-text E0, this is a 10.6% latency cost and 0.3% peak-allocation increase in a cold one-query smoke. Persistent reuse is therefore the economic gate: the one-time 7.12-s encoding cost must be amortized across repeated queries before native PRA can improve this frontier.

6.3 Residency Frontier

At 32K backing tokens under the balanced four-consumer profile, HOT PRA retained 0.50 MiB, hybrid retained 0.375 MiB, and layer-streamed PRA retained 0.125 MiB. Thus layer streaming reduced peak PRA detail by 4× relative to HOT for the same 128 selected tokens. In the controlled decomposition at 64K, full-context execution reached 601.0 MiB while balanced layer-streamed native PRA reached 89.125 MiB. The 85.2% reduction follows directly from holding selected evidence and query length fixed; it is a memory-accounting result, not evidence that 64K model quality is preserved.

6.4 Prefetch Interaction

For the 32K balanced hybrid condition, no PRA prefetch took 155.83 ms per controlled pass. Independent one-worker PRA prefetch took 133.25 ms, a 14.5% reduction. Coordinated prefetch on AirLLM's weight executor took 142.56 ms, 8.5% below no prefetch but 7.0% above independent prefetch. "Coordinated" is therefore not synonymous with "faster": a shared serial queue can prevent thread multiplication yet lose useful overlap. A production controller should choose between independent I/O and queue coordination from observed storage parallelism rather than platform labels.

7 What the Results Do and Do Not Establish

The measurements establish a clean ownership boundary and a feasible physical lifecycle. They show that a bounded selected context does not inherit backing context's K/V growth in the accounting model. They also show

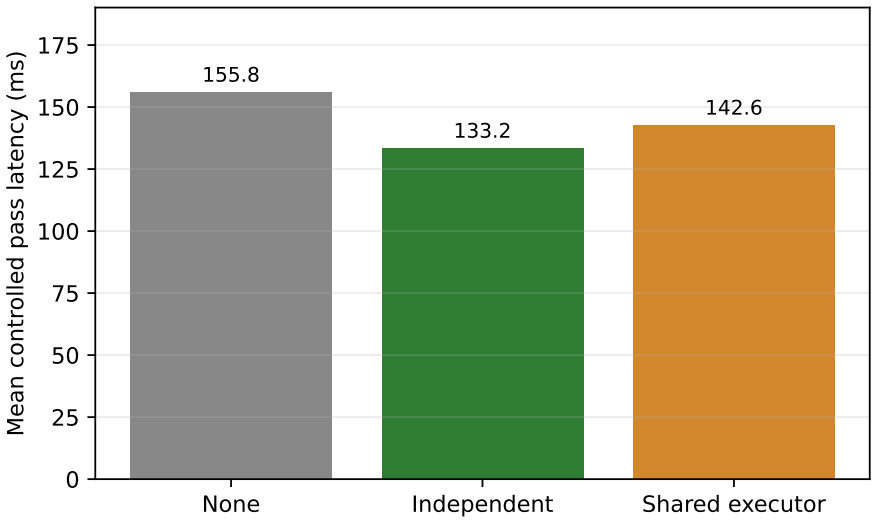

Figure 3: Controlled 32K/hybrid balanced-profile latency. Sharing AirLLM's one-worker executor avoids another thread but queues PRA behind weight reads.

Table 4: Evidence status at this revision.

Claim	Status	Evidence boundary
AirLLM delegates CUDA forward/generate to HF	Established	Pinned source audit
AirLLM/MLX runs layer-streamed TinyLlama	Established	Live M5 E0 run
Selected text preserves tested answer prefix	Established	Four live conditions, one example
PRA K/V can follow a layer-local lifecycle	Established	Tests plus 119-row controlled sweep
Native HF parameter placement bridge	Established	Unit tests and live disabled parity
Native AirLLM/PRA mechanism	Established	Live CUDA smoke; one synthetic case
Large-model-beyond-VRAM advantage	Open	Requires CUDA host and natural tasks

that AirLLM's weight-streaming cost can dominate short-prompt E0 inference, and that using one executor for two I/O streams may be inferior to bounded independent prefetch.

They do not yet establish that native PRA improves answer quality, TTFT, or peak VRAM for a model that cannot fit on the test accelerator. The small CUDA smoke closes AirLLM disabled, injected-but-disabled, reference encoding, and native consumption. A scale claim still requires a conventional HF baseline, repeated decode, HOT versus layer-streamed native PRA, matched source/query geometry, and frozen selected intervals on a larger natural cohort.

8 Product Contract

The current product row is deliberately conservative:

Engine	Platform	Default	Native candidate	Evidence tier
AirLLM	macOS/MLX	E0 selected text	unavailable	live smoke
AirLLM	CUDA/HF	E0 selected text	opt-in E1	live mechanism smoke

The runtime exposes `airllm` only as an embedded provider. Native flags must be explicit. No `QUALITY_MAX` product claim follows from this small calibration; profiles remain candidates until natural-task validation.

9 Limitations and Next Experiments

The M5 cohort is one synthetic retrieval example and one Llama-family model. Its MLX path differs architecturally from AirLLM's HF path. The controlled model uses explicit bandwidth and compute parameters and cannot estimate kernel-level attention cost or language quality. The host was under unrelated memory load, so available-memory deltas are supporting diagnostics only.

The next decisive experiment is not another hook microbenchmark. On a newer CUDA host, freeze selected intervals once and compare E0 selected text with E2 native K/V for cold one-shot, warm repeat, multi-query same resource, and shared-resource reuse. Report answer F1/EM, gold log probability, visible and native tokens, weight and PRA bytes, TTFT, inter-token latency, peak memory decomposition, and cache hits. Then repeat on a model materially larger than VRAM. A sparse-MoE run is useful only after dense-model correctness; expert selection and PRA routing must remain separately attributed.

10 Related Work

AirLLM [1] belongs to layer/expert streaming and offload systems. FlexGen [3] jointly schedules parameters, activations, and K/V across GPU, CPU, and disk for throughput-oriented constrained inference. Transformers provides the architecture and cache semantics reused by the integration described here [2]. PagedAttention and vLLM [4] address fragmentation and sharing of sequential serving K/V; their scheduler-visible block abstraction differs from AirLLM's layer-streamed parameter lifecycle. Retrieval and prompt compression move selected text into the ordinary prefix. PRA instead studies query-selected non-prefix native state. The distinguishing combination here is parameter streaming plus semantic K/V selection, with the two policies remaining independently measurable.

11 Conclusion

AirLLM and PRA target different memory terms and can share an HF execution path without sharing policy. The integration requires more than calling two APIs: PRA-wrapped parameter names must be reconciled with AirLLM shards, request device identity cannot be read from meta-resident embeddings, teardown must prevent selected-detail leakage, and prefetch queues must be measured rather than assumed. Live MLX E0 results show substantial visible-token reduction but also expose weight streaming as the dominant latency floor. Controlled results show a 4× PRA peak-residency reduction from layer-local streaming and a prefetch tradeoff between independent overlap and shared-queue discipline. A live CUDA smoke closes parameter-placement, disabled-parity, reference-encoding, and native-consumption gates, while showing a 10.6% cold native latency cost. The remaining scientific gate is natural quality and memory validation at a scale where both weight and context virtualization are necessary.

Reproducibility

Scripts are in `experiments/paper6_6_airllm`. Raw JSON, figures, and the source capability audit are under `docs/papers/shared/results/paper6_6_airllm` and the paper's `figures` directory. Unit tests cover provider negotiation, selective parameter-key remapping, lifecycle ordering, HOT reuse, teardown, and memory decomposition. Every live artifact records package versions, hardware, model, and its evidence tier.

References

[1] G. Lyongavin et al. *AirLLM: Run large language models on low-end GPUs.* Pinned source repository, 2026. `https://github.com/lyogavin/airllm`.

[2] T. Wolf et al. Transformers: State-of-the-art natural language processing. *EMNLP: System Demonstrations*, 2020.

[3] Y. Sheng et al. FlexGen: High-throughput generative inference of large language models with a single GPU. *ICML*, 2023.

[4] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. *SOSP*, 2023.